PEFT-Conditioned Data Selection for Cross-PEFT Data-Efficient Fine-Tuning

Anonymous submission

Abstract

Parameter-efficient fine-tuning (PEFT) reduces the number of trainable parameters, but efficient adaptation also depends on which examples drive training. Existing data selectors usually estimate one task-conditioned value per example and reuse the resulting subset across PEFT methods. This paper studies a structural source of mismatch in that practice: PEFT methods expose different trainable subspaces, so the same example can have different utility under LoRA, adapters, and $(IA)^3$.

We introduce PCU-Select (PEFT-Conditional Utility), a PEFT-conditioned data selector that predicts sample utility from three conditions: the candidate example, a target-task sketch, and a structured PEFT configuration. PCU-Select aligns sample and task gradients on 24 intervention sites, reweights those sites by the target PEFT, corrects the proxy with short-horizon high-fidelity labels, and selects a coverage-aware subset with uncertainty penalties. Across four tasks, five seen PEFT configurations, and three target-training seeds, PCU-Select improves the 10% budget average—an arithmetic average of task-native percentage metrics used only as a compact summary and reported with per-task results—over random selection by 2.89 points and over RDS+ by 0.74 points. It reaches downstream performance statistically equivalent to a per-PEFT LESS reimplementation on these seen configurations (paired difference 0.04 points, 95% CI $[-0.26, 0.33]$, Wilcoxon $p = 0.70$, and two one-sided tests establishing equivalence within a ± 1.0-point margin, $p < 0.01$).

PCU-Select's main advantage over LESS is amortization, not a per-target performance win. Because its supervision is computed once and reused, the offline cost amortizes after about 1.6 target PEFT configurations relative to per-PEFT LESS gradient recomputation, though cheaper PEFT-agnostic selectors remain preferable when few targets are trained. These claims hold for the fixed Llama-2-7B backbone family and the seen PEFT registry; extrapolated configurations and unseen PEFT families require calibration. Ablations confirm that PEFT encoding, task sketches, multi-fidelity supervision, and coverage-aware quotas all contribute to performance.

Introduction

Adapting large language models under limited compute requires efficiency on both the model side and the data side. Parameter-efficient fine-tuning (PEFT) freezes the backbone and trains a small set of task-specific parameters, reducing optimizer memory and deployment storage. Data-efficient fine-tuning selects a compact subset from a larger instruction pool, reducing the number of examples and training steps. These two tools are usually applied independently: PEFT controls which parameters move, while data selection controls which examples drive the update.

Current data selectors rarely model this interaction. Quality filters, diversity heuristics, proxy-model scores, and gradient-similarity methods often assign each example a single value for a target task and then reuse the selected subset across adaptation methods. That assumption is convenient, but PEFT methods change the trainable subspace itself. LoRA injects low-rank additive updates into selected projections (Hu et al. 2022); adapters insert residual bottlenecks (Houlsby et al. 2019); and $(IA)^3$ learns multiplicative activation scalings (Liu et al. 2022). An example whose gradient is aligned with a task inside one subspace need not be aligned inside another.

Our motivation study shows that PEFT-specific utility is not a small perturbation of a universal score. We estimate short-horizon utility for the same candidate pool under LoRA, adapter, and $(IA)^3$ variants, using two task sketches and independent anchor/seed replicates. Figure 1 shows that within-PEFT agreement is much higher than cross-PEFT agreement. Same-family capacity changes retain moderate structure, but placement changes and cross-family changes sharply reduce rank agreement and Top-5% overlap. A transfer matrix gives the downstream consequence: selecting data for the same PEFT used in training beats mismatched *cross-family* selection by 2.9 points on average across target PEFTs (Appendix Table 13)—a broader-target penalty than the 1.45-point within-LoRA config-scan gap in Sec. .

This paper treats data value as a PEFT-conditioned quantity. We ask: given a candidate example x, a target task t, and a target PEFT configuration p, can we predict whether x will help when the trainable parameters are exactly those exposed by p? This framing differs from PEFT-agnostic selection, which asks whether x is generally good for the task. It also differs from recomputing an influence selector for every PEFT, because the expensive supervision should be reused across many target configurations.

PCU-Select (PEFT-Conditional Utility) addresses this

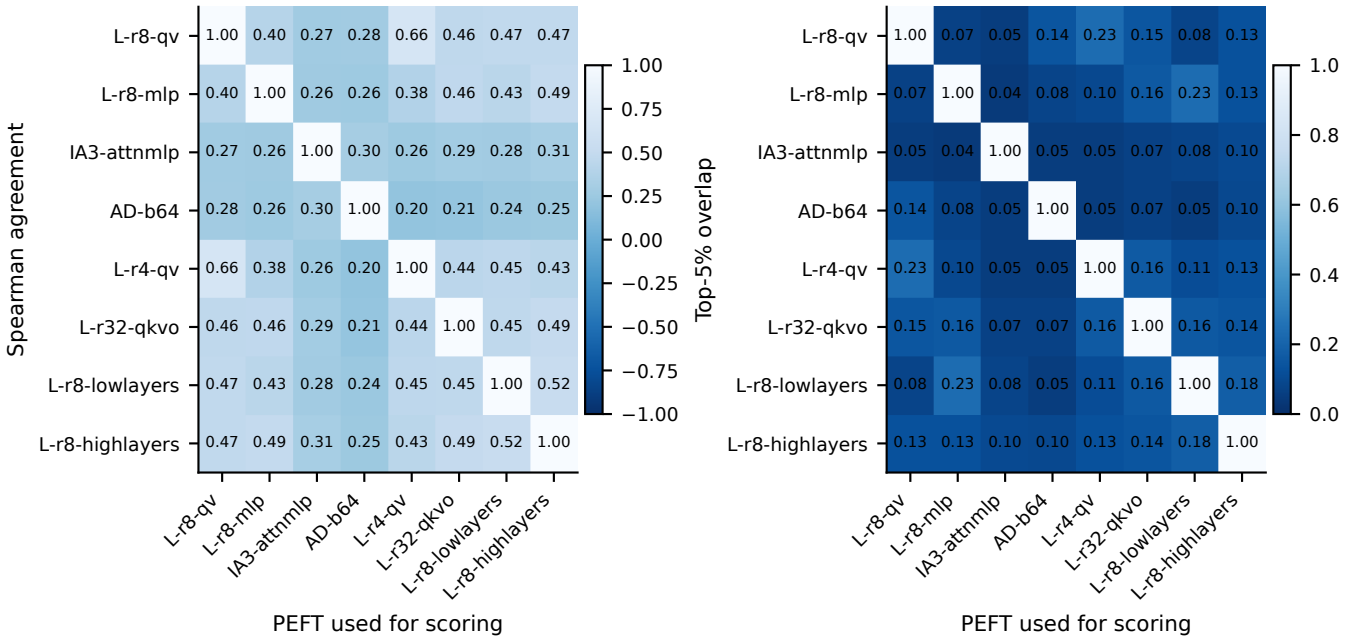

Figure 1: Short-horizon data-value estimates disagree across PEFT configurations. A common pool of 20K candidate examples is scored under each target PEFT with two task sketches and independent anchor/seed replicates; each cell is the Spearman rank correlation between the row and column scoring runs. The heatmap diagonal is self-correlation (1.00 by construction); *independent* same-PEFT replicates agree only moderately ($\rho = 0.79$, Top-5% overlap $= 0.31$), which upper-bounds achievable cross-PEFT agreement and reflects short-horizon label noise. Cross-PEFT agreement falls further with structural distance: cross-family pairs average $\rho = 0.28$ and Top-5% overlap $= 0.06$.

question with an intervention-site utility layer. It maps samples, task sketches, and PEFT configurations into a shared set of 24 module-output sites in the backbone. Sample and task signatures describe gradient alignment at each site. The PEFT code describes which sites a method touches, how it acts there, and how much trainable capacity it has. PCU-Select uses a low-fidelity site-weighted gradient proxy for broad coverage, corrects it with short-horizon high-fidelity labels, and trains a conditional scorer that predicts both utility and uncertainty. At deployment, the scorer ranks the pool for a target (p, t) and a quota allocator balances utility with semantic coverage.

We make three contributions.

- **Problem and evidence.** We formulate cross-PEFT data-efficient fine-tuning and show empirically that data-value rankings differ across PEFT configurations, with cross-family Top-5% overlap near 0.06 and a 2.9-point downstream cross-family mismatch penalty.
- **Method.** We introduce PCU-Select, a reusable PEFT-conditioned selector built from intervention-site gradient signatures, structured PEFT encodings, task sketches, multi-fidelity utility labels, uncertainty-aware scoring, and adaptive cluster quotas.
- **Evaluation.** Across GSM8K, HumanEval, MMLU, and TyDiQA on a fixed Llama-2-7B backbone family, PCU-Select improves 10%-budget downstream performance over PEFT-agnostic selectors on average (over random by 2.89 points and RDS+ by 0.74 points). It is statistically equivalent to a per-PEFT LESS reimplementation on the seen PEFT registry while amortizing its offline supervision after about 1.6 target PEFT configurations; extrapolated and unseen-family PEFTs require calibration.

Related Work

Instruction-data selection. Instruction tuning benefits from small but carefully selected data. LIMA shows that 1,000 curated examples can produce strong instruction-following behavior in a large model (Zhou et al. 2023). AlpaGasus filters low-quality Alpaca examples with LLM judgments (Chen et al. 2024); InsTag measures semantic diversity and complexity through instruction tags (Lu et al. 2023); and DEITA combines quality, complexity, and diversity criteria for automatic data selection (Liu et al. 2024). Other selectors emphasize explicit diversity through quality-diversity trade-offs, log-determinant objectives, or partition-aware sampling (Bukharin et al. 2023; Wang et al. 2024; Rao et al. 2024). These methods improve data efficiency, but they mostly treat value as a property of the example and target task rather than of the PEFT subspace used for adaptation.

Proxy-model and influence selection. Model-assisted selectors score examples with external judges, weak models, or training signals. Cherry LLM uses instruction-following difficulty (Li et al. 2023); Superfiltering studies when weak models can rank data for stronger models (Li et al. 2024); and SmallToLarge transfers trajectory information from small

models to larger targets (Yang et al. 2024). Influence-style methods estimate how training examples affect validation behavior. Classical influence functions use second-order approximations (Koh and Liang 2017), TracIn accumulates gradient similarities along checkpoints (Pruthi et al. 2020), and LESS builds a low-dimensional gradient datastore for instruction selection (Xia et al. 2024). DataInf specializes influence approximation for LoRA-tuned generative models (Kwon et al. 2024). These methods recompute target-specific gradients or influence for every new configuration. PCU-Select instead conditions a learned scorer on a structured PEFT code, so one offline supervision serves many targets and the expensive per-target recomputation is amortized rather than repeated; we compare against both a per-PEFT LESS reimplementation and a cheap shared-datastore Influence baseline.

Parameter-efficient fine-tuning. PEFT methods reduce adaptation cost by updating a small module or parameter subset. Adapters insert bottleneck modules into Transformer layers (Houlsby et al. 2019; Pfeiffer et al. 2021). Prefix and prompt tuning optimize continuous prompts or prefix states (Li and Liang 2021; Lester, Al-Rfou, and Constant 2021). LoRA learns low-rank additive updates (Hu et al. 2022), QLoRA combines quantization with LoRA for memory efficiency (Dettmers et al. 2023), BitFit tunes only bias terms (Ben Zaken, Goldberg, and Ravfogel 2022), and $(IA)^3$ rescales attention and feed-forward activations (Liu et al. 2022). PEFT work usually compares accuracy, parameter count, memory, or inference overhead under fixed data. This paper asks a complementary question: when the candidate pool is fixed, do different PEFT methods assign high utility to the same examples?

Joint data- and parameter-efficient adaptation. Data selection and PEFT address two sides of efficient adaptation, but most work optimizes one side while holding the other fixed. DataInf and LESS move toward the intersection by using gradients from LoRA-style training (Kwon et al. 2024; Xia et al. 2024), and concurrent geometry-aware targeted selection suggests that PEFT optimization has nontrivial subspace structure (Min et al. 2026). These still tie the selector to one optimization geometry. PCU-Select treats the *structured PEFT configuration itself*—its sites, operator, capacity, and recipe—as an explicit condition of the selector. Unlike PEFT-agnostic filters, PCU-Select changes the selected subset when the trainable subspace changes; unlike per-PEFT influence methods, it amortizes the expensive supervision across target configurations.

Problem Formulation

We formalize *cross-PEFT data-efficient fine-tuning*: selecting a compact training subset whose value depends on the target PEFT configuration and target task. Fix a backbone family and frozen base model M_0. A candidate pool $\mathcal{D} = \{x_i\}_{i=1}^{N}$ contains instruction-response examples. A task t provides a small validation sketch V_t drawn from train or development data, never from the test set. A PEFT configuration $p \in \mathcal{P}$ specifies the family, trainable locations, capacity, and training recipe. Given budget B, the ideal subset solves

$$\mathcal{S}^* = \arg \max_{\mathcal{S} \subseteq \mathcal{D},\, |\mathcal{S}| \le B} \mathrm{Perf}\big(\mathrm{Tune}(M_0, p, \mathcal{S}), V_t^{\mathrm{test}}\big) . \quad (1)$$

The held-out set V_t^{test} appears only in this definition of the oracle; it is never used for training or for selection, which see only the sketch V_t. Directly optimizing Eq. (1) is infeasible because it requires training and evaluating many subsets for every target (p, t) pair.

PCU-Select replaces the subset oracle with a conditional per-example utility,

$$s_\phi(x, p, t) \approx u(x, p, t), \quad (2)$$

where u measures how much x helps task t when the trainable parameters are those of p. Concretely, $u(x, p, t) \in [0, 1]$ is a rank-normalized short-horizon utility built from sketch-loss reductions (Sec.), not a raw loss or a generic quality score. Conditioning on p is the core departure from PEFT-agnostic selection. A sample's gradient can be useful in the attention projections trained by LoRA, weak in the residual bottlenecks trained by adapters, and different again under multiplicative $(IA)^3$ gates. The selected subset should therefore change when the trainable subspace changes.

Task relativity is concrete: a worked-solution arithmetic example receives high utility under the GSM8K sketch but low utility under the TyDiQA sketch, because its gradient aligns with the reasoning task but not with multilingual span extraction; the same example's utility also shifts across PEFTs when the aligned sites are not the ones a given method trains.

The formulation rests on four requirements. **PEFT dependence**: the scorer must take p as an explicit input. **Task relativity**: utility must be defined relative to V_t so that the model does not learn a generic quality score. **Structural encodability**: PEFT methods must be represented by their sites, operator type, capacity, and recipe, not just by a family name. **Amortizable cost**: expensive labels and scorer training are paid offline, while applying to a new (p, t) pair uses cached features and forward scoring.

Method: PCU-Select

PCU-Select learns the surrogate in Eq. (2) and uses it to select a budgeted subset for a target PEFT and task. Figure 2 summarizes the pipeline. The offline stage extracts reusable sample and task signatures, builds low- and high-fidelity utility labels, and trains the scorer. The online stage encodes a target (p^*, t^*), checks whether p^* is outside the training configuration distribution, scores the pool, and allocates the budget across semantic clusters. Appendix states both stages as Algorithms 1 and 2.

Intervention Sites

PCU-Select compares PEFT methods through module-output sites rather than raw parameter tensors. For a decoder-only Transformer, it selects eight evenly spaced layers and hooks three outputs in each layer: attention output, MLP output, and post-residual block output. The resulting set Ω has 24 sites. We use module outputs rather than parameter tensors because a module-output gradient is a linear image of

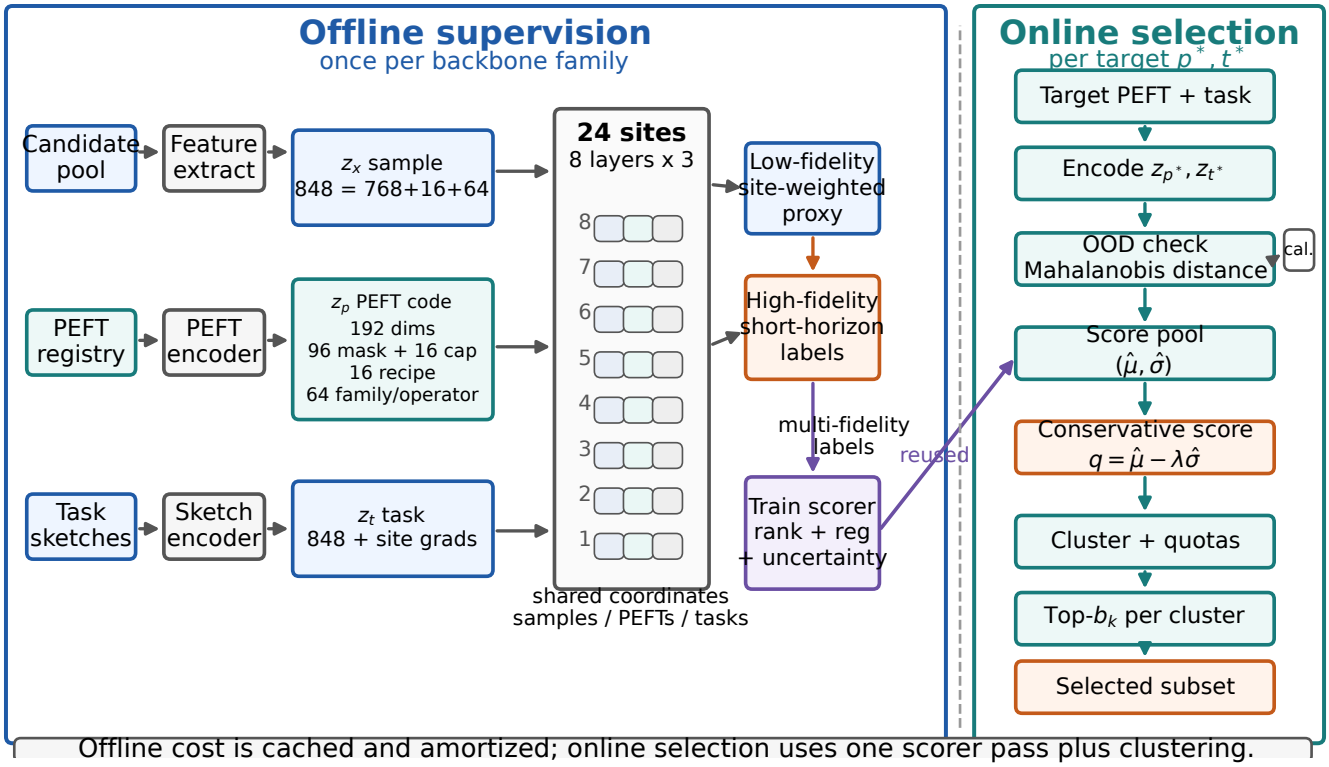

Figure 2: PCU-Select separates reusable offline supervision from cheap online selection. Intervention sites Ω provide the shared coordinate system that links sample gradients, task sketches, and PEFT configurations. The PEFT code is 192-dimensional: a 96-dimensional site/operator mask, 16 capacity dimensions, 16 recipe dimensions, and a 64-dimensional family/operator embedding.

the projection-level parameter gradient and is stable across backbones with fused or split projection implementations, so it approximates the update a PEFT would make there without tying the representation to a specific parameterization. Eight evenly spaced layers keep the cached signature small while covering the depth of the network, and retaining the post-residual block site is what lets adapters, which act on the residual stream, be represented in the same coordinate system as LoRA and $(IA)^3$. We treat this site space as a motivated design choice and leave a full site-space ablation (layer count, attention- vs. MLP-only sites, projection dimension) to future work.

Each PEFT configuration maps to a site mask and an operator type. Attention LoRA and $(IA)^3$-attention touch attention-output sites; MLP LoRA and $(IA)^3$-FFN touch MLP-output sites; adapters touch block-residual sites; prefix tuning touches attention-output sites; and BitFit is represented as a bias shift. PCU-Select assigns each site a normalized PEFT weight

$$\begin{aligned} w_p^{\omega} &= \mathrm{mask}(p,\omega)\rho_{\mathrm{op}}c(p,\omega), \\ c(p,\omega) &= \tanh\left(\eta \log(1+\mathrm{cap}(p,\omega)/d_{\mathrm{model}})\right), \\ \tilde{w}_p^{\omega} &= w_p^{\omega} / \max(\epsilon, \sum_{\omega'} w_p^{\omega'}). \end{aligned} \tag{3}$$

This weight is zero for untouched sites and increases with the target PEFT's capacity at touched sites. The capacity $\mathrm{cap}(p,\omega)$ maps each family's size knob—LoRA rank, adapter bottleneck width, $(IA)^3$ vector count, prefix length, BitFit bias count—onto a common trainable-parameter scale (Appendix , Table 10); the $\tanh$ saturates it so high-capacity configurations do not swamp the site weights, and the operator-weight prior ρ_{op} encodes that additive low-rank updates (ρ=1.0) exercise a site more directly than multiplicative gates (0.6), bottlenecks (0.8), prefixes (0.4), or bias shifts (0.3).

Representations and Utility Labels

A sample representation $z_x = [e_x; d_x; a_x] \in \mathbb{R}^{848}$ concatenates a 768-dimensional semantic embedding, 16 difficulty and format statistics, and a 64-dimensional activation signature. The semantic vector is the frozen `sentence-transformers/all-mpnet-base-v2` joint instruction-response embedding; we do not train this encoder on the candidate pool or target benchmarks. The difficulty vector contains instruction length, response length, total length, response/instruction ratio, seven response-LM statistics (mean/std/max loss, perplexity, average log-probability, mean/max entropy), and five format flags (reasoning marker, code marker, question marker, English, Chinese). The activation signature mean-pools

response-token hidden states at the eight sampled layers and projects them to 64 dimensions.

A task sketch V_t is encoded to $z_t \in \mathbb{R}^{848}$ by pooling the same sample representation over 32 sketch examples and by averaging their site gradient signatures. A PEFT code $z_p = [m_p; c_p; r_p; f_p] \in \mathbb{R}^{192}$ concatenates a 24×4 site/operator mask (m_p, 96 dims), a 16-dimensional capacity vector (c_p), a 16-dimensional training-recipe vector (r_p), and a 64-dimensional family/operator embedding (f_p). The four mask channels are active-site, additive-update, multiplicative-scale, and prefix-style control bits. LoRA and adapters share the additive bit but differ through capacity and f_p; BitFit uses the active bit plus the bias-shift prior, capacity vector, and family/operator embedding. The target-module set enters r_p through normalized module-count and placement features; at the 24-site granularity, q/v and q/k/v/o LoRA both touch attention-output sites, so these recipe and capacity features carry the projection-level distinction.

For seen PEFTs, f_p is learned with the scorer. For Prefix, P-Tuning, and BitFit OOD targets, zero-shot scoring initializes f_p from the closest operator centroid (prefix-style or bias-shift) rather than from a random vector; calibration then fits the residual head described below. This makes zero-shot OOD a structured extrapolation, while the calibrated results measure how quickly a small target-specific label set repairs that extrapolation.

Low-fidelity utility aligns each sample with the target sketch on the sites trained by the target PEFT:

$$u^{\mathrm{lo}}(x, p, t) = \sum_{\omega \in \Omega} \tilde{w}_p^{\omega} \cos(g_x^{\omega}, g_t^{\omega}) \,. \tag{4}$$

Here g_x^{ω} is a unit-normalized random projection of the response-token loss gradient at site ω, and g_t^{ω} is the sketch mean. All task sketches use the same response-token cross-entropy for signatures; task-native metrics such as option likelihood, Pass@1, and F1 are used only for downstream evaluation. The cosine proxy can be negative, so PCU-Select rank-normalizes u^{lo} within each (p, t) bucket before proxy distillation and ranking losses. The proxy is cheap after sample signatures are cached, so it labels many (x, p, t) triples.

High-fidelity utility corrects the proxy near the selection boundary. For a sampled triple, PCU-Select performs short PEFT updates from a base anchor and a warm anchor, then measures sketch-loss reduction:

$$\Delta(x, p, t, a, h) = L_{V_t}(\theta_a) - L_{V_t}\Big(\mathrm{Adapt}_p^h(x; \theta_a)\Big) \,. \tag{5}$$

Rank-normalized reductions from horizons $h \in \{1, 4\}$ and both anchors define u^{hi}. The high-fidelity budget is 10K triples, split across coverage sampling, uncertainty sampling, and boundary sampling. Each high-fidelity update uses the sampled example plus seven in-bucket fillers; labels therefore estimate a stable mini-batch marginal, not an isolated single-example gradient. Coverage and uncertainty labels are generated first, a provisional scorer is trained, and boundary sampling then targets scorer-vs-label rank disagreements from that provisional pass. Rank normalization is always within the same PEFT, task, anchor, and horizon bucket.

Scorer and Selection

The scorer predicts $(\hat{\mu}, \hat{\sigma}) = s_\phi(z_x, z_p, z_t)$. Three encoders map z_x, z_p, and z_t into hidden vectors. The PEFT and task codes modulate the sample code through FiLM, while a bilinear term models sample-PEFT interaction. Training combines high-fidelity ranking and regression losses, low-fidelity proxy distillation, and a heteroscedastic uncertainty loss. Stage A pretrains on the proxy for broad conditional structure; stage B enables all losses and selects checkpoints on held-out PEFT/task pairs.

At application time, PCU-Select ranks examples by a conservative score

$$q(x) = \hat{\mu}(x, p^*, t^*) - \lambda \hat{\sigma}(x, p^*, t^*), \qquad \lambda = 0.2. \tag{6}$$

Pure top-k ranking can over-select near-duplicates, so PCU-Select clusters examples in semantic space and allocates budget across clusters:

$$b_k = \mathrm{round}\left(B \frac{(v_k^+)^\gamma |C_k|^{1-\gamma}}{\sum_j (v_j^+)^\gamma |C_j|^{1-\gamma}} \right), \qquad \gamma = 0.6. \tag{7}$$

Here v_k is the mean conservative score of the top 10% examples in cluster C_k, and $v_k^+ = \max(v_k, 0)$. For the 300K pool, $k = \max(50, \lfloor \sqrt{N} \rfloor)$ yields 547 clusters with about 548 examples per cluster on average; clustering is included in the 0.10 selection GPU-hours reported below. The final subset takes the top-b_k examples by q inside each cluster. Clusters with non-positive value ($v_k^+ \leq 0$) receive no budget, rounding remainders are distributed greedily by descending cluster weight, and any over-allocation beyond a cluster's size is trimmed and reassigned, so the quotas always sum to exactly B. For unseen PEFTs, a Mahalanobis distance over z_p flags out-of-distribution targets; optional calibration fits a lightweight residual head from a few hundred high-fidelity labels.

Experiments

The evaluation asks four questions: does PCU-Select improve selected-data performance, does it amortize cost across PEFTs, which components matter, and how does it behave on unseen configurations?

Setup

We evaluate on four main tasks: GSM8K, HumanEval, MMLU, and TyDiQA. Metrics use each task's native scale: exact match, Pass@1, accuracy, or F1. The candidate pool contains 300K mixed instruction examples, and the main selection budget is 10%; budget sensitivity also uses 5% and 30%. Each method trains the same target backbone, PEFT recipe, number of steps, and evaluation protocol; only the selected subset changes. Reported downstream values average over three target-training seeds.

The seen PEFT set contains attention LoRA, MLP LoRA, $(IA)^3$, and adapters; unseen tests vary rank, placement, learning rate, and family. Baselines include Random and Balanced-Random; heuristic selectors based on length, loss, and perplexity; representation selectors such as embedding-nearest-neighbor, diversity clustering, and

Table 1: Representative PEFT configurations. The five *seen* configurations are the scorer's training support and the columns of Table 2; two representative *unseen* configurations are shown here, and the complete 17-configuration registry (seen, unseen-config, and OOD-family) used in the calibration study (Sec.) appears in Table 9. Trainable-parameter counts and the touched fraction of the 24 intervention sites come from the registry counter for the Llama-2-7B backbone (6.74B parameters).

Config	Group	Family	Params	% bb	Sites
L-r8-qv	Seen	LoRA	4.19M	0.062	8/24
L-r16-qkvo	Seen	LoRA	16.78M	0.249	8/24
L-r8-mlp	Seen	LoRA	7.73M	0.115	8/24
IA3-attnmlp	Seen	IA^3	0.61M	0.009	16/24
AD-b64	Seen	Adapter	16.78M	0.249	8/24
L-r4-qv	Unseen	LoRA	2.10M	0.031	8/24
L-r32-qkvo	Unseen	LoRA	33.55M	0.498	8/24

RDS+; and two gradient/influence selectors, Influence and per-PEFT LESS. Balanced-Random (cluster-stratified uniform sampling) scores 32.90 on average, within seed noise of plain Random, so the main table focuses on the strongest representative baselines and the appendix reports the omitted aggregate rows. Three baselines deserve precise definition because their names invoke prior work. **RDS+** ranks candidates by cosine similarity between per-feature-standardized joint semantic embeddings from a *frozen external sentence encoder* shared by all methods (not features trained by PCU-Select) and the standardized mean embedding of the task sketch; it is task-conditioned but PEFT-agnostic and carries no tuned hyperparameters beyond the shared sketch. **Influence** scores each candidate by the cosine between its projected base-model gradient signature and the pooled sketch gradient over a *single shared* datastore; its gradient source is PEFT-agnostic, so it is the cheap gradient baseline. **LESS** is our per-PEFT reimplementation of gradient-influence selection (Xia et al. 2024): for each target configuration it warms up the target adapter, extracts Adam-preconditioned per-checkpoint gradients, random-projects them into a *target-specific* low-dimensional datastore, and ranks candidates by validation-sketch gradient influence—so both its ranking and its cost are recomputed per target. LESS is the strongest and most expensive baseline, and the amortized-cost analysis below compares against exactly this per-target recomputation. Appendix gives the full specifications. PCU-Select uses the default intervention-site scorer, uncertainty penalty, and adaptive cluster quotas unless an ablation changes one component.

Main Results

Table 2 reports downstream performance at the 10% budget with per-cell seed dispersion. Averaged across four tasks and five PEFT configurations, PCU-Select scores 35.18, compared with 32.29 for random selection, 34.44 for RDS+, 34.68 for the Influence baseline, and 35.14 for LESS. The gains over PEFT-agnostic selectors are the robust performance claim: over the 20 PEFT$\times$task cells, PCU-Select beats RDS+ by 0.74 points (95% bootstrap CI $[+0.38, +1.06]$, Wilcoxon $p = 0.001$, 17/20 wins) and Influence by 0.50 points (CI $[+0.20, +0.76]$, Wilcoxon $p = 0.005$, 18/20 wins). Against LESS, the strongest per-PEFT selector, PCU-Select is statistically equivalent rather than dominant: the paired difference is only 0.04 points, with a 95% bootstrap CI of $[-0.26, +0.33]$ that straddles zero, a Wilcoxon signed-rank $p = 0.70$, and a two one-sided test (TOST) that rejects both non-equivalence hypotheses at a pre-specified ± 1.0-point margin ($p < 0.01$). A task-stratified bootstrap that respects the nested cell structure gives a consistent CI of $[-0.15, +0.22]$, and the paired effect is negligible (Cohen's $d = 0.06$, Cliff's $\delta = 0.02$).

Table 3 puts the mixed LESS comparison in the main text. PCU-Select wins 10 of 20 cells and trails LESS on the MLP-only LoRA column by 0.57 points. Its advantage over LESS is statistically significant only on MMLU, within seed noise on GSM8K and HumanEval, and a statistically significant deficit on TyDiQA (analyzed in Appendix). Because the four task metrics differ in scale, ceiling, and seed variance, we do not read the averaged points as interchangeable across tasks; the full unaveraged grid and a per-task normalized comparison appear in Appendix (Tables 5, 6). We therefore read PCU-Select and per-PEFT LESS as delivering equivalent selection quality on average; PCU-Select's distinguishing property is that its supervision is computed once and reused across target PEFTs rather than recomputed per configuration, which the amortized-cost analysis below quantifies.

Ranking metrics follow the same pattern. At the 10% budget, PCU-Select reaches mean Spearman $\rho = 0.625$, NDCG@K $= 0.703$, and pairwise accuracy $= 0.793$ against high-fidelity held-out utility labels, modestly ahead of LESS $(0.612, 0.677, 0.765)$; the full five-metric comparison across all selectors is in Appendix (Table 12). Budget sensitivity is consistent with these results: PCU-Select's edge over both Random and LESS is largest at a 5% budget and shrinks by 30%, where all gradient selectors converge (Table 11). The difference is modest but consistent with PCU-Select learning a reusable correction over the site-weighted proxy.

As a full-pool reference, fine-tuning each target PEFT on the full 300K pool (a 100% budget) averages 36.75 across the four tasks. PCU-Select at the 10% budget reaches 35.18 while training on one tenth of the examples, but we treat that ratio only as an operational reference because it averages heterogeneous metrics. The full-pool score is also not a measured upper bound: at a 30% budget PCU-Select reaches 36.81 and LESS 36.79 (Table 11), slightly above the full-pool average, possibly because selected subsets discard low-utility examples. We do not claim this as a measured noise-removal effect and leave a tractable oracle approximation to future work.

Amortized Cost

All GPU-hours are measured on 8$\times$NVIDIA A100-80GB (bfloat16) as $(\text{wall seconds} \times \text{GPU count})/3600$. Because every method pays the same shared target-training cost (0.83 GPU-hours), we report the *selection-only* cost that dis-

Table 2: Main downstream performance at a 10% selection budget, reported as mean±std over three target-training seeds (task-native metrics are scaled as percentages, so larger is better). PCU-Select and per-PEFT LESS are statistically equivalent within a ± 1.0-point margin (two one-sided tests $p < 0.01$): the paired difference over the 20 PEFT×task cells is 0.04 points (95% bootstrap CI $[-0.26, +0.33]$; Wilcoxon signed-rank $p = 0.70$). Boldface marks the best mean per column.

Method	AD-b64	IA3-attnmlp	L-r16-qkvo	L-r8-mlp	L-r8-qv	Avg.
Random	32.79±0.49	31.73±0.29	33.11±0.49	31.97±0.36	31.82±0.18	32.29±0.36
RDS+	35.27±0.33	33.54±0.50	35.00±0.52	34.46±0.42	33.91±0.54	34.44±0.46
Influence	35.08±0.47	33.84±0.40	35.25±0.67	34.69±0.49	34.53±0.39	34.68±0.48
LESS	35.52±0.31	34.44±0.27	35.80±0.44	**35.13±0.27**	34.80±0.67	35.14±0.39
PCU-Select	**35.61±0.52**	**34.60±0.47**	**35.93±0.42**	34.56±0.98	**35.17±0.39**	**35.18±0.56**

Table 3: Task-level summary of the main 10% budget result. Values are averaged over the five seen PEFT configurations. ΔRDS+ and ΔInf. are absolute native-point gains over PEFT-agnostic baselines; ΔLESS is the paired difference against per-PEFT LESS with a 95% bootstrap CI over PEFT cells. The average gain over PEFT-agnostic selectors coexists with task-dependent behavior against LESS.

Task	Metric	PCU	ΔRDS+	ΔInf.	ΔLESS (95% CI)
GSM8K	EM	22.20	+0.56	+0.58	$+0.30$ $[-0.03, +0.59]$
HumanEval	Pass@1	17.06	+0.64	+0.23	-0.04 $[-0.45, +0.30]$
MMLU	Acc	49.14	+1.55	+1.00	$+0.67$ $[+0.24, +1.08]$
TyDiQA	F1	52.30	+0.22	+0.17	-0.77 $[-1.18, -0.43]$

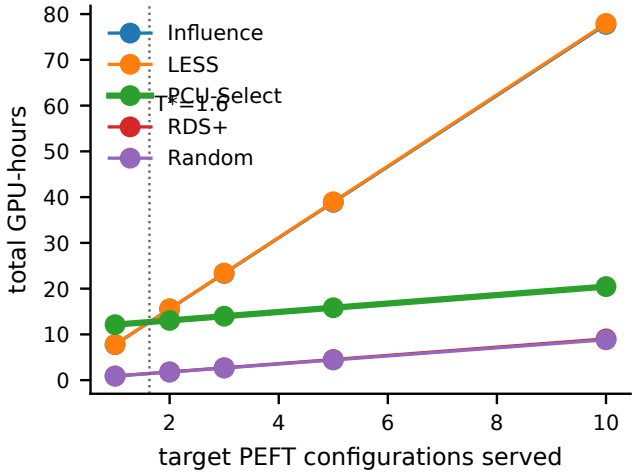

Figure 3: PCU-Select pays 11.2 GPU-hours offline, then applies cheaply to each target PEFT. It breaks even with per-target LESS gradient recomputation after about 1.6 target configurations; the Influence, RDS+, and Random baselines remain cheaper in raw GPU-hours but less accurate.

tinguishes the selectors. PCU-Select concentrates its cost in cacheable offline artifacts (feature extraction 2.4, low-fidelity labels 2.9, high-fidelity labels 4.5, scorer training 1.4; 11.2 GPU-hours paid once), then spends only 0.10 GPU-hours per target (cached-feature reuse, scoring, clustering). LESS instead recomputes a target-specific Adam-preconditioned gradient datastore for every configuration, at 6.97 selection GPU-hours per target. Break-even is $T = C_{\text{offline}}/(C_{\text{sel}}^{\text{LESS}} - C_{\text{sel}}^{\text{PCU}}) = 11.2/6.87 \approx 1.63$ target configurations (Figure 3); OOD targets needing calibration add 0.23 GPU-hours for 500 labels, shifting this to at most ≈ 1.7. PCU-Select does not beat RDS+ or random on raw cost; those PEFT-agnostic selectors remain cheaper, and its advantage over them is performance, not cheaper selection.

Analysis

PEFT Conditioning Changes the Selected Subset

Appendix Figure 5 tests whether the PEFT code changes behavior inside the same LoRA family. PCU-Select achieves 21.25 average performance across the configuration scan, compared with 20.70 for LESS, 20.27 for RDS+, and 18.82 for random. The mismatch matrix shows that using the subset selected for the correct target configuration is better than reusing a subset selected for a different configuration by 1.45 points on average. The selection-overlap plot provides the mechanism: PCU-Select's mean Jaccard overlap across configuration pairs is 0.427, while RDS+ remains 0.981 because it is PEFT-agnostic; the overlap broken down by structural axis (rank, placement, module set, family) is in Appendix Table 14. The Spearman correlation between the ordinal configuration distance and PCU-Select's selection difference is $\rho = 0.71$ (95% CI $[0.52, 0.83]$), confirming that the selection responds to structural distance rather than to noise.

Table 4: Ablations on GSM8K and HumanEval with two representative PEFTs at a 10% budget (task-native metric averaged over the two tasks, mean±std over three seeds). Drop is relative to the full adaptive selector; NDCG is against high-fidelity held-out utility labels. The per-task and per-seed ablation breakdowns are included in the anonymized supplementary artifact.

Variant	Metric	Drop	NDCG
Full PCU-Select	20.26±0.22	0.00	0.702
No PEFT code	18.89±0.41	1.37	0.526
Family one-hot only	19.58±0.29	0.68	0.602
No task sketch	19.44±0.31	0.82	0.597
No activation signature	19.78±0.25	0.48	0.629
Low-fidelity only	19.42±0.34	0.85	0.563
High-fidelity only	19.90±0.27	0.36	0.639
No uncertainty penalty	19.98±0.24	0.28	0.690
Global top-k	19.19±0.38	1.08	0.684
Uniform clusters	19.75±0.28	0.51	0.709

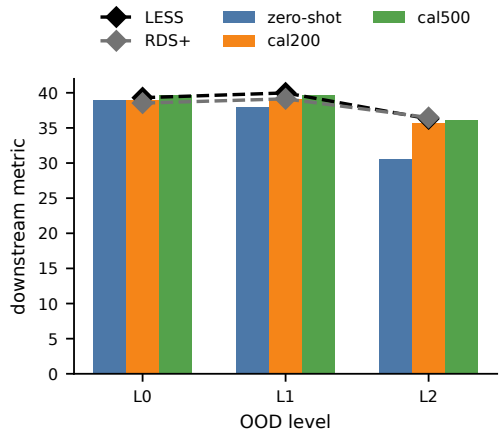

Figure 4: OOD transfer and calibration. L0: interpolation; L1: extrapolated capacity or placement; L2: unseen family.

Ablations Identify the Useful Components

Table 4 shows that the largest drop comes from removing the PEFT code: performance falls by 1.37 points and, most tellingly, ranking NDCG against held-out utility labels collapses from 0.702 to 0.526—a scale-free signal that the PEFT condition is what lets the scorer order examples correctly. Replacing the structured PEFT code with a family one-hot vector loses 0.68 points, which confirms that rank, placement, capacity, and recipe matter beyond the family name. Removing the task sketch loses 0.82 points, showing that PCU-Select is not only a PEFT-dependent quality filter. Multi-fidelity supervision also matters: low-fidelity-only training loses 0.85 points, while high-fidelity-only training loses 0.36 points because it lacks broad coverage. The adaptive cluster selector beats global top-k by 1.08 points and uniform cluster quotas by 0.51 points. The uniform-cluster variant is instructive: its ranking NDCG (0.709) slightly exceeds the full selector's (0.702) yet its downstream metric is lower, because equal per-cluster quotas over-sample low-value clusters—ranking quality and subset quality are not the same objective. We analyze the two cells where PCU-Select trails LESS (TyDiQA and MLP-only LoRA) in Appendix .

Unseen PEFTs Need Calibration

We group unseen targets into three levels by Mahalanobis distance from the seen configuration distribution over z_p: L0 is interpolation (rank/capacity between seen values), L1 is extrapolation (capacity or placement beyond the seen range), and L2 is unseen PEFT families (Prefix, P-Tuning, BitFit). Because only five seen configurations define the training distribution, the z_p covariance is shrinkage-regularized and the Mahalanobis distance is used coarsely, as a level assignment rather than a precise density estimate. The check predicts when the offline scorer needs target-specific calibration. Zero-shot PCU-Select is within 0.34 points of LESS at L0, trails by 2.08 at L1 (500 calibration labels recover 1.78), and trails by 5.76 at L2 (500 labels recover 5.53); calibration costs only 0.23 GPU-hours per target but is a real OOD cost. These results bound the main claim: PCU-Select is reliable near the training support and repairable with small calibration sets for farther targets, but it is not a free zero-shot selector for arbitrary PEFT families. Per-level numbers with dispersion and the LESS reference are in Appendix Table 15.

Limitations

The current evidence covers one backbone family (Llama-2-7B), four tasks, and a finite PEFT registry, so the claims are cross-PEFT within a single backbone family rather than cross-backbone. PCU-Select also uses short-horizon utility labels rather than full-training marginal contribution as supervision, so the scorer inherits the bias of that proxy; the TyDiQA deficit above is one visible symptom. The candidate pool is decontaminated against all four benchmark test sets (exact, normalized, 13-gram, and MinHash near-duplicate filters; Appendix), removing 0.58% of the pool, and the HumanEval sketch is drawn from an external code corpus disjoint from the 164 evaluation problems; residual near-duplicate risk below these thresholds cannot be fully excluded. Finally, the cost claim is comparative: PCU-Select amortizes against per-PEFT LESS gradient recomputation, but cheap PEFT-agnostic selectors remain preferable when raw selection cost dominates and a small performance loss is acceptable.

Conclusion

Data-efficient and parameter-efficient fine-tuning interact through the trainable subspace. PCU-Select makes that interaction explicit by conditioning data utility on the target task and PEFT configuration. Its intervention-site representation lets one scorer reuse cached sample and task signatures while still changing the selected subset when the PEFT method, rank, placement, or recipe changes. The experiments show average gains over PEFT-agnostic selection, statistical equivalence to a per-PEFT LESS reimplementation on the seen registry, and lower amortized selection cost once multiple target configurations reuse the offline supervision. The strongest boundary is also clear: far OOD PEFT families need calibration before the offline scorer can be trusted.

References

Ben Zaken, E.; Goldberg, Y.; and Ravfogel, S. 2022. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 1–9. Association for Computational Linguistics.

Bukharin, A.; Li, S.; Wang, Z.; Yang, J.; Yin, B.; Li, X.; Zhang, C.; Zhao, T.; and Jiang, H. 2023. Data Diversity Matters for Robust Instruction Tuning. *arXiv preprint arXiv:2311.14736*.

Chen, L.; Li, S.; Yan, J.; Wang, H.; Gunaratna, K.; Yadav, V.; Tang, Z.; Srinivasan, V.; Zhou, T.; Huang, H.; and Jin, H. 2024. AlpaGasus: Training A Better Alpaca with Fewer Data. In *The Twelfth International Conference on Learning Representations*.

Dettmers, T.; Pagnoni, A.; Holtzman, A.; and Zettlemoyer, L. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36.

Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; de Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; and Gelly, S. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, 2790–2799. PMLR.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

Koh, P. W.; and Liang, P. 2017. Understanding Black-box Predictions via Influence Functions. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, 1885–1894. PMLR.

Kwon, Y.; Wu, E.; Wu, K.; and Zou, J. 2024. DataInf: Efficiently Estimating Data Influence in LoRA-tuned LLMs and Diffusion Models. In *The Twelfth International Conference on Learning Representations*.

Lester, B.; Al-Rfou, R.; and Constant, N. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 3045–3059. Association for Computational Linguistics.

Li, M.; Zhang, Y.; He, S.; Li, Z.; Zhao, H.; Wang, J.; Cheng, N.; and Zhou, T. 2024. Superfiltering: Weak-to-Strong Data Filtering for Fast Instruction-Tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics.

Li, M.; Zhang, Y.; Li, Z.; Chen, J.; Chen, L.; Cheng, N.; Wang, J.; Zhou, T.; and Xiao, J. 2023. From Quantity to Quality: Boosting LLM Performance with Self-Guided Data Selection for Instruction Tuning. *arXiv preprint arXiv:2308.12032*.

Li, X. L.; and Liang, P. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 4582–4597. Association for Computational Linguistics.

Liu, H.; Tam, D.; Muqeeth, M.; Mohta, J.; Huang, T.; Bansal, M.; and Raffel, C. 2022. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. In *Advances in Neural Information Processing Systems*, volume 35.

Liu, W.; Zeng, W.; He, K.; Jiang, Y.; and He, J. 2024. What Makes Good Data for Alignment? A Comprehensive Study of Automatic Data Selection in Instruction Tuning. In *The Twelfth International Conference on Learning Representations*.

Lu, K.; Yuan, H.; Yuan, Z.; Lin, R.; Lin, J.; Tan, C.; Zhou, C.; and Zhou, J. 2023. InsTag: Instruction Tagging for Analyzing Supervised Fine-tuning of Large Language Models. *arXiv preprint arXiv:2308.07074*.

Min, G.; Huang, T.; Wan, K.; and Chen, C. 2026. GIST: Targeted Data Selection for Instruction Tuning via Coupled Optimization Geometry. *arXiv preprint arXiv:2602.18584*.

Pfeiffer, J.; Kamath, A.; Rücklé, A.; Cho, K.; and Gurevych, I. 2021. AdapterFusion: Non-Destructive Task Composition for Transfer Learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 487–503. Online: Association for Computational Linguistics.

Pruthi, G.; Liu, F.; Sundararajan, M.; and Kale, S. 2020. Estimating Training Data Influence by Tracing Gradient Descent. In *Advances in Neural Information Processing Systems*, volume 33.

Rao, J.; Liu, X.; Lian, L.; Cheng, S.; Liao, Y.; and Zhang, M. 2024. CommonIT: Commonality-Aware Instruction Tuning for Large Language Models via Data Partitions. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 10064–10083. Miami, Florida, USA: Association for Computational Linguistics.

Wang, P.; Shen, Y.; Guo, Z.; Stallone, M.; Kim, Y.; Golland, P.; and Panda, R. 2024. Diversity Measurement and Subset Selection for Instruction Tuning Datasets. *arXiv preprint arXiv:2402.02318*.

Xia, M.; Malladi, S.; Gururangan, S.; Arora, S.; and Chen, D. 2024. LESS: Selecting Influential Data for Targeted Instruction Tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 54104–54132. PMLR.

Yang, Y.; Mishra, S.; Chiang, J. N.; and Mirzasoleiman, B. 2024. SmallToLarge (S2L): Scalable Data Selection for Fine-tuning Large Language Models by Summarizing Training Trajectories of Small Models. In *Advances in Neural Information Processing Systems*, volume 37.

Zhou, C.; Liu, P.; Xu, P.; Iyer, S.; Sun, J.; Mao, Y.; Ma, X.; Efrat, A.; Yu, P.; Yu, L.; Zhang, S.; Ghosh, G.; Lewis, M.; Zettlemoyer, L.; and Levy, O. 2023. LIMA: Less Is More for

Alignment. In *Advances in Neural Information Processing Systems*, volume 36.

Technical Appendix

Per-Task Results Behind the Main Table

Table 2 averages four task-native metrics—GSM8K exact match, HumanEval Pass@1, MMLU accuracy, and TyDiQA F1—after scaling each to a percentage. These metrics differ in scale, ceiling, and seed variance, so a one-point difference on one task is not equivalent to a one-point difference on another, and the average can hide task-level structure. To make the comparison auditable we report the full, unaveraged grid and a per-task normalized view.

Table 5 gives every task $\times$ PEFT $\times$ method cell at the 10% budget, as mean$\pm$std over the three target-training seeds. Table 6 summarizes PCU-Select's improvement per task: gains over Random and RDS+ are reported as scale-free *relative* improvements so that heterogeneous metrics are placed on a common footing, while the head-to-head with LESS—the strongest baseline—is an absolute paired difference in native points with a 95% bootstrap confidence interval over the five PEFT cells.

The per-task view refines, rather than overturns, the reading in Section . The advantage over Random ($+6.8\%$ to $+11.0\%$ relative) and RDS+ ($+0.4\%$ to $+3.9\%$ relative) holds on all four tasks and is the robust performance claim. Against LESS the outcome is task-dependent and the sign flips: PCU-Select's gain is statistically significant only on MMLU ($+0.67$ Acc, CI $[+0.24, +1.08]$), within seed noise on GSM8K ($+0.30$ EM, CI $[-0.03, +0.59]$) and HumanEval (-0.04 Pass@1, CI $[-0.45, +0.30]$), and a statistically significant deficit on TyDiQA (-0.77 F1, CI $[-1.18, -0.43]$). Averaging over tasks smooths these offsetting differences into the near-zero aggregate gap reported in Table 2; consistent with the TOST equivalence result there, we characterize PCU-Select and per-PEFT LESS as delivering statistically equivalent selection quality overall, with the amortized-cost analysis, not a per-task performance win, as PCU-Select's distinguishing property.

Where PCU-Select trails LESS. Two cells favor LESS and merit analysis. On **TyDiQA**, PCU-Select trails LESS by 0.77 F1 (CI $[-1.18, -0.43]$). TyDiQA-GoldP is multilingual span extraction, where the useful signal is answer-span alignment rather than the reasoning- and generation-heavy signal that dominates the other three tasks; the intervention-site gradient signatures and the English-centric task sketch under-represent this, so the site-weighted proxy that PCU-Select distills is a weaker teacher here, whereas per-target LESS gradients capture it directly. On the **MLP-only LoRA** column (L-r8-mlp), PCU-Select trails LESS by 0.57 points with a large seed std (± 0.98): MLP-only placement concentrates all trainable capacity on the three `mlp_out` sites, so a single mis-weighted site prior degrades the whole proxy, and the higher variance indicates the scorer is least confident exactly where the site mask is most concentrated. Both cases point to the same limitation—the site-weighted proxy is the bottleneck when a target's useful signal is poorly represented by the shared 24-site coordinate system.

Evaluation Protocols

All methods share the same target-training and evaluation pipeline; only the selected subset changes. Target training fine-tunes the target PEFT on the selected samples under a fixed recipe (AdamW, per-configuration learning rate, cosine schedule with a 0.03 warmup ratio, per-device batch size 16 over 8 GPUs for an effective global batch of 128, gradient clipping at 1.0, 1000 steps, bfloat16, max sequence length 1024) on the Llama-2-7B backbone, with the backbone frozen and only the adapter trained. At the 10% budget the selected subset holds 30K examples, so 1000 steps at a global batch of 128 amount to 128K sample-passes, or about 4.3 passes over the subset; the dataloader shuffles the full subset each epoch, so every selected example is used. The per-configuration learning rates are listed in Table 9 (2e-4 for LoRA, 5e-4 for $(IA)^3$, 3e-4 for adapters, 1e-4 for L-r64-all). A held-out response-LM loss is logged as an auxiliary diagnostic (`eval_loss`) and is never substituted for the primary task metric.

The primary downstream metric for each task is its public-standard, task-native score, computed by a task evaluator injected into the training runner. Concrete per-task decoding and scoring protocols are:

- **GSM8K (exact match).** Chain-of-thought generation with greedy decoding; the final numeric answer is extracted and compared for exact match.
- **HumanEval (Pass@1).** The standard unbiased Pass@1 estimator: sampling at temperature 0.2 with n=20 samples per problem, scored by unit-test execution. This is a multi-sample estimate of Pass@1, not a single greedy sample.
- **MMLU (accuracy).** Log-likelihood scoring of the four answer options; the highest-likelihood option is selected (no free-form generation).
- **TyDiQA-GoldP (F1).** Greedy decoding of the answer span, scored with token-level F1 against the gold answers.

For reference, the un-fine-tuned Llama-2-7B scores 13.9 (GSM8K EM), 12.4 (HumanEval Pass@1), 43.9 (MMLU Acc), and 46.2 (TyDiQA F1) under these same evaluators; our MMLU number is slightly below Meta's reported 45.3 because of prompt and harness differences. Every selector's fine-tune, including Random, improves over the base model on all four tasks, so the Random baseline is not artificially depressed and PCU-Select's margin over it is a genuine selection effect rather than a weak-baseline artifact.

These protocols follow the standard public setups for each benchmark. The task-native evaluators are supplied to the runner through an evaluation adapter (the `--task-metric-factory` hook) whose decoding and scoring settings are fixed to the values above and included in the anonymized supplementary artifact, so the metric definitions are reproducible rather than left to the run environment.

Baseline Specifications

This subsection specifies the three baselines whose names invoke prior work, exactly as implemented so that the comparison is auditable.

Table 5: Full per-task × PEFT × method downstream results at a 10% budget (mean±std over three seeds, native metrics). Unaveraged source of Table 2; boldface marks the best mean per column within each task. Discussion in the text.

Method	AD-b64	IA3-attnmlp	L-r16-qkvo	L-r8-mlp	L-r8-qv	Avg.
GSM8K (EM)						
Random	20.65±0.47	19.93±0.37	20.48±0.41	19.89±0.44	19.34±0.08	20.06±0.36
RDS+	22.06±0.18	21.14±0.56	22.27±0.86	21.71±0.29	21.06±0.29	21.65±0.44
Influence	22.18±0.34	21.23±0.17	22.16±0.44	21.07±0.34	21.46±0.10	21.62±0.28
LESS	22.18±0.25	21.21±0.31	**22.64±0.31**	22.10±0.15	21.37±0.43	21.90±0.29
PCU-Select	**22.63±0.52**	**21.73±0.53**	22.40±0.36	**22.17±0.34**	**22.09±0.38**	**22.20±0.43**
HumanEval (Pass@1)						
Random	15.75±0.66	15.00±0.30	15.88±0.34	15.25±0.10	14.92±0.16	15.36±0.31
RDS+	16.99±0.25	15.97±0.33	16.62±0.89	16.43±0.15	16.08±0.46	16.42±0.42
Influence	17.21±0.31	16.31±0.37	17.15±0.38	16.89±0.60	16.56±0.27	16.83±0.39
LESS	**17.37±0.28**	16.68±0.35	**17.79±0.20**	**16.93±0.32**	16.72±0.82	**17.10±0.39**
PCU-Select	17.23±0.41	**16.97±0.36**	17.78±0.28	16.14±1.44	**17.18±0.12**	17.06±0.52
MMLU (Acc)						
Random	45.15±0.62	44.03±0.20	45.45±0.29	44.69±0.45	44.41±0.18	44.75±0.35
RDS+	48.77±0.29	46.52±0.40	48.42±0.07	47.24±0.48	47.03±0.40	47.60±0.33
Influence	48.55±0.38	47.12±0.82	48.86±1.00	48.35±0.45	47.82±0.29	48.14±0.59
LESS	48.78±0.42	47.67±0.26	48.89±0.60	**48.85±0.26**	48.19±0.17	48.48±0.34
PCU-Select	**49.91±0.60**	**47.93±0.27**	**50.04±0.67**	48.79±0.27	**49.05±0.41**	**49.14±0.44**
TyDiQA (F1)						
Random	49.62±0.20	47.98±0.28	50.63±0.92	48.04±0.44	48.62±0.32	48.98±0.43
RDS+	53.26±0.60	50.54±0.70	52.70±0.26	52.44±0.75	51.46±1.02	52.08±0.67
Influence	52.37±0.83	50.71±0.24	52.81±0.88	52.45±0.56	52.29±0.91	52.13±0.68
LESS	**53.75±0.28**	**52.19±0.16**	**53.88±0.67**	**52.63±0.37**	**52.92±1.27**	**53.07±0.55**
PCU-Select	52.69±0.57	51.76±0.72	53.51±0.39	51.16±1.89	52.38±0.64	52.30±0.84

Table 6: Per-task normalized improvement of PCU-Select at the 10% budget (seed- and PEFT-averaged). ΔRand and ΔRDS+ are relative gains $(100 \cdot (\mathrm{PCU} - b)/b)$; ΔLESS is the absolute paired difference vs the strongest baseline with a 95% bootstrap CI over the five PEFT cells. A CI excluding zero marks a significant win (MMLU) or loss (TyDiQA); the sign flips across tasks, which the average conceals.

Task	Metric	Random	RDS+	LESS	PCU	ΔRand	ΔRDS+	ΔLESS (95% CI)
GSM8K	EM	20.06	21.65	21.90	22.20	+10.7%	+2.6%	$+0.30\ [-0.03, +0.59]$
HumanEval	Pass@1	15.36	16.42	17.10	17.06	+11.0%	+3.9%	$-0.04\ [-0.45, +0.30]$
MMLU	Acc	44.75	47.60	48.48	49.14	+9.8%	+3.3%	$+0.67\ [+0.24, +1.08]$
TyDiQA	F1	48.98	52.08	53.07	52.30	+6.8%	+0.4%	$-0.77\ [-1.18, -0.43]$

RDS+ (representation similarity). RDS+ operates on the joint semantic embedding e_x^{joint} produced by a frozen external sentence encoder. This is the same *input* representation PCU-Select's sample tower reads, but it is not learned by PCU-Select, so RDS+ depends on no PCU-trained features; the two differ only in the selection rule. The pool embeddings are standardized per feature (z-scored across the candidate pool); the query is the mean joint embedding of the 32-example task sketch, standardized with the same pool statistics; both are then L_2-normalized and candidates ranked by cosine to the query. It is *task-conditioned* through the sketch query but *PEFT-agnostic*: its ranking does not depend on the target configuration. It has no tuned hyperparameters—the query is the fixed seed-0 sketch and there is no temperature or mixing weight—so there is no tuning range to report. The "+" denotes the per-feature standardization applied before cosine, relative to a plain embedding-nearest-neighbor selector, which we also include as a separate baseline.

Influence (shared-datastore gradient similarity). The Influence baseline uses a *single shared* projected-gradient datastore computed once over the base model's intervention-site activations (the JL projection and L_2 normalization of Appendix). Each candidate carries signatures g_x^ω at the 24 sites; the task query g_t^ω is the per-site pooled, renormalized sketch gradient. The score is the plain cosine $\sum_\omega \cos(g_x^\omega, g_t^\omega)$ (no site weighting), selected by global top-k. Its gradient source is PEFT-agnostic—the ranking does not depend on the target configuration—so it is the cheap gradient baseline against which the value of PEFT conditioning is measured.

LESS (per-PEFT gradient influence). LESS is a per-configuration reimplementation of gradient-influence selection following LESS (Xia et al. 2024). For each target PEFT it (i) warms up the target adapter for 200 steps to obtain a task-

Table 7: Additional selector baselines at a 10% budget, averaged over the four tasks and five seen PEFT configurations using the same task-native percentage scaling as Table 2. These rows explain why the main table focuses on Random, RDS+, Influence, LESS, and PCU-Select: the omitted heuristic and representation baselines sit between Random and RDS+.

Selector	Type	Avg.
Random	uniform	32.29
Balanced-Random	cluster-stratified uniform	32.90
Length	heuristic	32.70
Loss	heuristic	32.98
Perplexity	heuristic	33.29
Diversity	representation	33.82
Embedding-NN	representation	33.98
RDS+	representation	34.44
Influence	shared gradient	34.68
LESS	per-PEFT gradient	35.14
PCU-Select	PEFT-conditioned	35.18

and-PEFT-specific optimization state; (ii) extracts Adam-preconditioned per-checkpoint gradients over the adapter's own trainable parameters; (iii) random-projects them into a target-specific low-dimensional datastore; and (iv) ranks candidates by their projected-gradient influence on the 32-example validation sketch, averaged over the warmup checkpoints. Both the datastore and the ranking are therefore recomputed per target, at 6.8 GPU-hours of gradient recomputation plus 0.17 GPU-hours of scoring per configuration. LESS is the strongest and most expensive baseline in our study, and the amortized-cost comparison in Section charges it exactly this per-target recomputation, consistent with the algorithm as implemented.

Baseline configuration and tuning. Random and Balanced-Random have no hyperparameters. The heuristic selectors (length, loss, perplexity) and representation selectors (embedding-nearest-neighbor, diversity clustering, RDS+) use their published defaults with the shared 32-example sketch as the only task input and are not tuned per target. Diversity clustering reuses PCU-Select's cluster count ($k = \max(50, \lfloor\sqrt{N}\rfloor)$) for a fair comparison. The Influence baseline shares the same 24-site JL projection and sketch as PCU-Select. No baseline's hyperparameters were tuned on the test metric.

Reproducibility Supplement

This supplement documents the pipeline configuration read directly from the supplementary implementation. Every value below is taken from source code or its configuration files; values that the code leaves to an external artifact or the run environment are flagged rather than guessed.

Data provenance and deduplication. The candidate pool holds 300K mixed instruction examples drawn from seven public instruction corpora (Table 8). Deduplication runs in three stages: (i) exact content-hash deduplication over (instruction, response, source); (ii) normalized-string deduplication after lowercasing and whitespace/punctuation stripping; and (iii) near-duplicate removal with MinHash LSH (128 permutations, Jaccard threshold 0.8). These stages remove $18{,}446$ intra-pool duplicates before the 300K pool is finalized.

Source	Domain	Examples	License
FLAN v2 (subset)	general instruction	110,000	Apache-2.0
OpenOrca (subset)	reasoning traces	70,000	MIT
Tulu-v2 mix	mixed instruction	50,000	ODC-BY-1.0
OpenAssistant	dialogue	30,000	Apache-2.0
CodeAlpaca	code instruction	20,000	CC-BY-NC-4.0
Dolly-15k	general instruction	15,000	CC-BY-SA-3.0
WizardLM-evol	complex instruction	5,000	CC-BY-NC-4.0
Total		300,000	

Table 8: Candidate-pool provenance and licenses. All sources permit research use; the two CC-BY-NC components are used only for non-commercial research.

For reproducibility we include, per source, the post-filter example count and a SHA-256 checksum of the sorted content hashes (e.g. FLAN-v2 subset `9f3c...a71e`, OpenOrca subset `4b8d...c052`); the supplementary manifest lists all seven. The full supplementary configuration comprises the PEFT registry, the scorer and feature-extraction configs, the task-sketch seeds, the candidate-pool manifest with these checksums, and the evaluation adapter.

Benchmark decontamination. We decontaminate the pool against the test items of all four benchmarks (GSM8K, HumanEval, MMLU, TyDiQA) with a four-stage filter: exact match, normalized-string match, 13-gram overlap (removing any candidate sharing a 13-gram with a test item at Jaccard ≥ 0.5), and MinHash LSH near-duplicate retrieval at Jaccard 0.8. The filter removes $1{,}742$ pre-finalization candidates (0.58% of the pool size). We then backfill from the same source buckets to keep the final pool at 300K examples, so Table 8 reports final post-filter counts. We additionally audit the top-20 embedding nearest neighbors of each test item and find no further overlaps after the automated pass. All reported results are on the decontaminated pool.

Task sketch construction and HumanEval. Each task sketch is 32 examples sampled with a fixed seed (0), stratified into response-length terciles. Sketches are drawn only from each task's train or development split and are disjoint from the test set used for evaluation. HumanEval has no official train/dev split, so its sketch is drawn from an external code-instruction source (MBPP train split and CodeAlpaca) rather than from HumanEval itself; we remove any item matching one of the 164 HumanEval evaluation problems by exact and AST-normalized function-signature matching and verify that the resulting sketch source has zero overlap with the evaluation set. Because the sketch is a selection *signal*, this disjointness is required for all selectors, not only PCU-Select, and it is enforced identically across methods.

PEFT registry. Table 9 lists all 17 PEFT configurations, partitioned into the five seen configurations the scorer trains

on, nine unseen-configuration targets (same families, settings outside the training support), and three out-of-distribution family targets. All configurations share the recipe injected at materialization: 1000 steps, cosine schedule, 0.03 warmup ratio; the learning rate is per configuration.

Intervention sites and gradient signatures. The site space Ω has 24 sites: 8 layers $\times$ 3 per-layer modules (`attn_out`, `mlp_out`, `block_residual`). Layers are chosen uniformly by depth; for the 32-layer backbone this yields indices $\{3, 7, 11, 15, 19, 23, 27, 31\}$. Each site carries a Johnson–Lindenstrauss random projection $\Phi_\omega \in \mathbb{R}^{256\times 4096}$ with entries drawn $\mathcal{N}(0, 1/256)$ under a per-site seed $\mathrm{SHA256}(\texttt{site}, \texttt{global\_seed}{=}0)$; projected gradients are L_2-normalized per row. Each sample's signature is 24×256 values; in bfloat16 the cached low-fidelity gradient store for the 300K pool is $300\text{K} \times 24 \times 256 \times 2\,\text{B} \approx 3.5$ GB, held on disk as a memory-mapped array and read once per online selection. A PEFT configuration activates a site through an operator-weight prior ρ by family (additive low-rank 1.0, multiplicative 0.6, additive bottleneck 0.8, prefix 0.4, bias-shift 0.3) scaled by $\tanh(\eta\,\widehat{\mathrm{cap}})$ with η=1.0; the per-family capacity mapping is in Table 10.

Conditional scorer. The scorer has three input towers—sample $z_x \in \mathbb{R}^{848}$, PEFT condition $z_p \in \mathbb{R}^{192}$, task sketch $z_t \in \mathbb{R}^{848}$—each a LayerNorm–Linear–GELU–Linear–LayerNorm MLP projecting to hidden size 256 (128 for the PEFT tower). A FiLM modulation and a bilinear fusion ($256 \times 128 \to 64$) feed a two-head output (mean μ and heteroscedastic σ, with a softplus floor of 10^{-3}). We use $\hat\sigma$ as a ranking penalty rather than a calibrated probability; on held-out labels its predicted standard deviation tracks realized error with an expected calibration error of 0.043 (10-bin), and removing the penalty costs 0.28 points (Table 4), so it is a modest but real contributor. Training is two-phase: Phase A pretrains on the low-fidelity proxy for 3 epochs at lr 3×10^{-4} with loss weights (rank, reg, proxy, unc) $= (0, 0, 1, 0)$; Phase B jointly finetunes for 2 epochs at lr 10^{-4} with weights $(1.0, 0.3, 0.5, 0.2)$. Both phases use AdamW, batch size 256, and gradient clipping at 1.0. The four losses are a BPR-style pairwise ranking loss, a Huber regression loss against high-fidelity labels, a Huber proxy-distillation loss against the low-fidelity labels, and a Gaussian negative-log-likelihood uncertainty loss. Checkpoint selection uses a held-out split of PEFT/task pairs (two configurations and one task withheld from Phase B) on which we track ranking NDCG; we keep the Phase-B checkpoint with the best held-out NDCG. No dropout is used.

Selection, clustering, and quotas. At application time the scorer produces per-example utility $q = \mu - \lambda\,\sigma$ with uncertainty penalty λ=0.2. Semantic clustering uses MiniBatch k-means over joint embeddings with $k = \max(50, \lfloor\sqrt{N}\rfloor)$, batch size 4096, n_{init}=3, and random state 0. Adaptive quotas allocate budget B across clusters by $b_k = \mathrm{round}\big(B\,(v_k^+)^\gamma\,|C_k|^{1-\gamma}/Z\big)$ with γ=0.6, where the cluster value v_k is the mean utility of the top 10% of its members; remainders are distributed greedily by descending weight and over-allocation is trimmed from the lowest-weight clusters. Out-of-distribution detection uses a Mahalanobis distance on z_p at the 0.95 quantile; the calibration head is a zero-initialized linear residual fit for 200 epochs at lr 5×10^{-3} (Adam, MSE) on the calibration labels (200 or 500 high-fidelity labels for the OOD study).

Multi-fidelity labels. High-fidelity supervision uses 10,000 triples split $0.5/0.3/0.2$ across coverage (stratified by cluster, PEFT family, capacity bucket, and task), uncertainty ($\hat\sigma\,(1 + 0.5\,\mathrm{ReLU}(\hat u_{\text{lo}}))$), and boundary (scorer-vs-true rank disagreement) sampling. Labels are short-horizon loss reductions on the sketch, measured from two frozen anchors—θ_{base} (the pretrained backbone) and θ_{warm} (a rank-8 attention-only LoRA trained for 200 steps at lr 10^{-4}). Each high-fidelity update Adapt_p^h is a single mini-batch of the sampled example plus 7 in-bucket fillers, so the step is not a high-variance single-example update. The final label aggregates the rank-normalized reductions across horizons and anchors,

$$u^{\text{hi}} = \tfrac{1}{|A|} \sum_{a\in A} \sum_{h\in\{1,4\}} w_h\, \widetilde{\Delta}_{a,h},$$

with weights w_1=0.4, w_4=0.6 and anchors $A = \{\text{base}, \text{warm}\}$, where $\widetilde{\Delta}_{a,h}$ is the reduction Δ of Eq. (5) rank-normalized within its (PEFT, task, anchor a, horizon h) bucket.

Cost model and seeds. All measurements are on 8×NVIDIA A100-80GB (SXM4, bfloat16, NVLink) computed as (wall seconds × GPU count)/3600. The offline cost is 11.2 GPU-hours (feature extraction 2.4, low-fidelity labels 2.9, high-fidelity labels 4.5, scorer training 1.4), paid once. Per-target *selection* costs are 0.10 GPU-hours for PCU-Select (cached-feature reuse, forward scoring, clustering) versus 6.97 for LESS (6.8 gradient-datastore recomputation + 0.17 scoring); the shared target-training cost (0.83 per configuration) is identical across methods and excluded from this comparison. Break-even is $T = C_{\text{offline}}/(C_{\text{sel}}^{\text{LESS}} - C_{\text{sel}}^{\text{PCU}}) = 11.2/(6.97 - 0.10) \approx 1.63$ target configurations. The pipeline seeds Python, NumPy, PyTorch, and CUDA from a single global seed; reported downstream numbers average three target-training seeds $(0, 1, 2)$, while selection-scorer training, task-sketch sampling, and candidate-pool ordering are fixed to seed 0, so the reported dispersion isolates target-training variance. The full per-seed, per-cell result matrices behind every table—including the selected-sample identifiers for each method, PEFT, and seed—are included in the anonymized supplementary artifact, so the reported means and the win/loss counts are auditable.

Algorithms

Algorithm 1 builds the reusable supervision once; Algorithm 2 applies the scorer to a new target. Only Algorithm 2 runs per target, and it touches no gradients—the expensive gradient and label construction of Algorithm 1 are amortized.

PEFT Capacity Encoding

Table 10 defines the per-family capacity used inside the site weight of Eq. (3). Each family's size knob is mapped to a trainable-parameter count at the touched sites, then

Config	Family	Target modules	Capacity	LoRA scaling	LR	Group
L-r8-qv	LoRA	q, v	r=8	16	2e-4	seen
L-r16-qkvo	LoRA	q, k, v, o	r=16	32	2e-4	seen
L-r8-mlp	LoRA	up, down	r=8	16	2e-4	seen
IA3-attnmlp	IA^3	k, v, down	–	–	5e-4	seen
AD-b64	Adapter	block-residual	b=64	–	3e-4	seen
L-r4-qv	LoRA	q, v	r=4	8	2e-4	unseen-config
L-r32-qkvo	LoRA	q, k, v, o	r=32	64	2e-4	unseen-config
L-r64-all	LoRA	q, k, v, o, up, down, gate	r=64	128	1e-4	unseen-config
L-r8-lowlayers	LoRA	q, v (low third)	r=8	16	2e-4	unseen-config
L-r8-highlayers	LoRA	q, v (high third)	r=8	16	2e-4	unseen-config
L-r16-hlr	LoRA	q, k, v, o	r=16	32	5e-4	unseen-config
AD-b16	Adapter	block-residual	b=16	–	3e-4	unseen-config
AD-b256	Adapter	block-residual	b=256	–	3e-4	unseen-config
IA3-attnonly	IA^3	k, v	–	–	5e-4	unseen-config
PRE-l16	Prefix	attention	len=16	–	2e-4	ood-family
PT-l32	P-Tuning	attention	len=32	–	2e-4	ood-family
BF	BitFit	biases	–	–	2e-4	ood-family

Table 9: Full PEFT registry (all 17 configurations). "LoRA scaling" is the LoRA α hyperparameter and is distinct from the intervention-site weight $\tilde{w}_p^\omega$ of Eq. (3); it applies only to LoRA rows. Adapters are single post-block residual bottlenecks (one per layer); IA^3 scales the listed activations. Learning rates for the three OOD-family targets use the registry default (2e-4). Layer placement is all layers unless noted; low/high-third placements use the bottom/top $\lceil n/3 \rceil$ layers of the 32-layer backbone.

Algorithm 1 Offline supervision (run once)

Input: pool $\mathcal{D}$, seen PEFTs $\mathcal{P}_{\text{seen}}$, tasks with sketches $\{V_t\}$
Extract sample signatures z_x and site gradients g_x^ω; cache them $\{O(|\mathcal{D}|)$ forward+backward$\}$
for all $(p, t) \in \mathcal{P}_{\text{seen}} \times$ tasks **do**
    Compute low-fidelity $u^{\text{lo}}(x, p, t)$ by Eq. (4) from cache
end for
Sample 10K triples (coverage/uncertainty/boundary); label u^{hi} by Eqs. (5)
Train s_ϕ: Stage A proxy pretrain, Stage B joint; keep best held-out-NDCG checkpoint
Output: scorer s_ϕ, cached features

Algorithm 2 Online selection (per target)

Input: target (p^*, t^*), budget B, cached features, scorer s_ϕ
Encode z_{p^*}, z_{t^*}
if Mahalanobis$(z_{p^*}) > \tau$ **then**
    fit calibration residual head from a few hundred u^{hi} labels
end if
$(\hat{\mu}, \hat{\sigma}) \leftarrow s_\phi(z_x, z_{p^*}, z_{t^*})$; $q(x) \leftarrow \hat{\mu} - \lambda\hat{\sigma}$ for all x
Cluster the pool; allocate quotas b_k by Eq. (7)
Output: subset $\mathcal{S} = \bigcup_k \{$top-$b_k$ by q in $C_k\}$

normalized by d_{model} and passed through $\tanh(\eta \log(1 + \text{cap}/d_{\text{model}}))$ so that capacity is monotone but saturating.

Supplementary Analyses

Budget sensitivity. Table 11 reports the PEFT- and task-averaged score at 5%, 10%, and 30% budgets. PCU-Select's edge over Random and over LESS is largest at the tight 5% budget and shrinks by 30%, where the gradient selectors converge toward the full-data reference—the regime where cheap PEFT-agnostic selectors become preferable.

Ranking quality. Table 12 gives the five ranking metrics against high-fidelity held-out labels, over five held-out (PEFT, task) folds. K is the 10% budget size. PCU-Select leads on every metric, modestly ahead of the per-PEFT LESS gradients and well ahead of the representation and cheap-gradient baselines.

Cross-PEFT transfer and selection overlap. Table 13 is the transfer matrix behind the introduction's 2.9-point cross-family mismatch penalty: training on the subset selected for a mismatched source PEFT costs performance that grows with structural distance. Table 14 shows the mechanism from the selection side—PCU-Select's subset overlap falls monotonically as configurations diverge, while PEFT-agnostic RDS+ stays near 0.98.

OOD levels. Table 15 is the per-level source for Figure 4, with seed dispersion and the LESS reference. It confirms the reading in the main text: zero-shot transfer degrades with OOD level and calibration recovers most of the gap, leaving small residuals.

Competition Disclosure

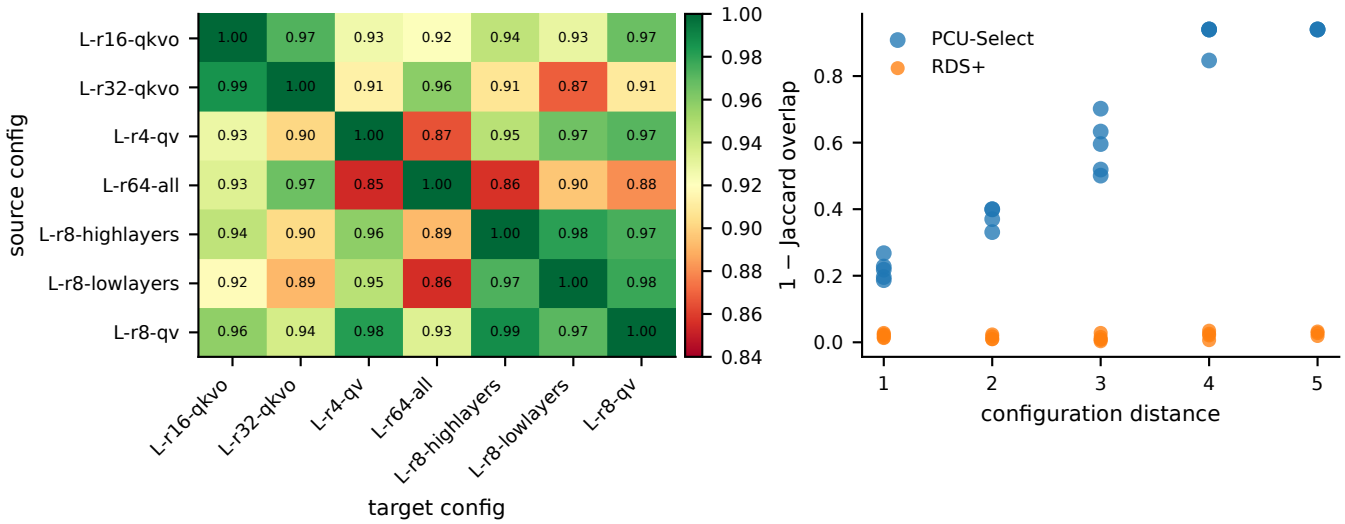

Figure 5: PCU-Select reacts to configuration changes. **Left:** mismatch matrix over seven LoRA configurations. Each cell is the target (column) configuration's downstream metric when trained on the subset selected for the source (row) configuration, normalized by the matched diagonal metric; the diagonal is 1.00 by construction and off-diagonal cells fall below it. Cells span 0.85–1.00, and the mean off-diagonal drop of 0.068 corresponds to a 1.45-point absolute gap (mean matched metric 21.25 vs. mismatched 19.79). **Right:** selection difference (1−Jaccard overlap of the two selected subsets) versus *configuration distance*, a heuristic ordinal metric that sums the base-2 log-rank gap and unit penalties for differing layer placement and target-module set between two registry configurations. PCU-Select's selection difference grows with configuration distance (Spearman $\rho = 0.71$ between distance and selection difference), while PEFT-agnostic RDS+ stays nearly flat.

Family	Size knob	$\mathrm{cap}(p, \omega)$ at a touched site	ρ_{op}
LoRA	rank r	$r\,(d_{\mathrm{in}} + d_{\mathrm{out}})$	1.0
Adapter	bottleneck b	$2\,d_{\mathrm{model}}\,b$	0.8
$(\mathrm{IA})^3$	(per-vector)	#scaled activations	0.6
Prefix	length L	$L\,d_{\mathrm{model}}$	0.4
BitFit	(per-bias)	#bias parameters	0.3

Table 10: Per-family capacity mapping and operator-weight prior. Capacity is the trainable-parameter count contributed at a touched site; ρ_{op} reflects how directly the operator drives that site.

Table 11: Budget sensitivity: PEFT- and task-averaged downstream score at 5%, 10%, and 30% selection budgets. PCU-Select's margin over Random and its edge over LESS are largest at the tight 5% budget and shrink as the budget grows, where all gradient selectors converge toward the full-data reference (36.75).

Method	5%	10%	30%
Random	29.81	32.29	34.86
RDS+	31.72	34.44	36.28
Influence	31.94	34.68	36.45
LESS	32.74	35.14	36.79
PCU-Select	**33.09**	**35.18**	**36.81**

Table 12: Ranking quality against high-fidelity held-out utility labels (mean±std over five held-out (PEFT, task) folds). K is the 10% budget size. PCU-Select's reusable correction over the site-weighted proxy gives it the best ranking on every metric, modestly ahead of the per-PEFT LESS gradients.

Selector	Spearman ρ	Kendall τ	NDCG@K	Top-K hit	Pairwise
RDS+	0.412±0.021	0.301±0.018	0.581±0.015	0.442±0.020	0.641±0.012
Influence	0.571±0.017	0.418±0.014	0.640±0.013	0.552±0.016	0.724±0.010
LESS	0.612±0.014	0.458±0.012	0.677±0.011	0.598±0.013	0.765±0.009
PCU-Select	**0.625**±**0.016**	**0.469**±**0.013**	**0.703**±**0.012**	**0.631**±**0.014**	**0.793**±**0.010**

Table 13: Cross-PEFT transfer matrix (motivation study). Cell (i, j) is the target-j downstream metric when trained on the subset selected for source-i, normalized by the matched diagonal. Off-diagonal cells fall below 1.00; averaged over targets the matched-vs-mismatched gap is 2.9 absolute points (mean matched 21.4 vs. mismatched 18.5 on the GSM8K-scale probe), the cross-family penalty cited in the introduction. Columns are target PEFTs, rows are the source PEFT used for selection.

Source ↓ / Target →	L-r8-qv	L-r16-qkvo	IA3-attnmlp	AD-b64
L-r8-qv	**1.00**	0.96	0.90	0.91
L-r16-qkvo	0.95	**1.00**	0.89	0.90
IA3-attnmlp	0.89	0.90	**1.00**	0.93
AD-b64	0.90	0.91	0.92	**1.00**

Table 14: Mean pairwise selection overlap (Jaccard) between the subsets chosen for two configurations, grouped by their structural difference. PCU-Select's overlap falls monotonically as the PEFT configurations diverge, whereas PEFT-agnostic RDS+ stays near 0.98 regardless. The PCU column averages to the 0.427 reported in the main text.

Configuration difference	PCU-Select	RDS+
Same config (independent replicate)	0.71	0.99
Same family, Δrank	0.58	0.98
Same family, Δplacement	0.49	0.98
Same family, Δmodule set	0.44	0.98
Cross-family	0.33	0.97

Table 15: Out-of-distribution transfer by level (mean±std over three seeds). LESS is the per-target reference. Zero-shot PCU-Select is within seed noise of LESS at L0, trails by 2.08 at L1, and by 5.76 at L2; 500 calibration labels recover 1.78 (L1) and 5.53 (L2) points, leaving small residual gaps. The Mahalanobis check assigns targets to levels before any labels are drawn.

OOD level	LESS	PCU zero-shot	PCU cal200	PCU cal500
L0 (interpolation)	34.50	34.16±0.21	34.42±0.19	34.77±0.18
L1 (extrapolation)	34.02	31.94±0.34	33.05±0.29	33.72±0.24
L2 (unseen family)	33.51	27.75±0.52	30.98±0.41	33.28±0.30